Temporary Vulnerable Sandbox Labs

CloudGuard now supports short-lived vulnerable labs for live demos and scanner validation.

The platform does **not** create or close full AWS/GCP cloud accounts on each click. Instead, it creates temporary vulnerable resources inside an already isolated sandbox account, project, or Kubernetes namespace, runs/queues the scanner, stores proof in the evidence/GRC workflow, and destroys the resources after scan completion or TTL expiry.

Why This Design

- AWS account creation/closure and GCP project deletion are asynchronous and not suitable for a 5-minute demo flow.
- Temporary resources can be tagged/namespaced, scanned, and destroyed quickly.
- The portal remains decoupled from scanner jobs: it stores requests and evidence, while `sandbox-lab-worker` performs provisioning/cleanup.
- Cloud resource deployment is disabled by default for AWS, GCP, and Kubernetes to prevent accidental cost or exposure.

API Flow

1. User clicks a demo action or calls `POST /api/sandbox-labs`.
2. A row is created in `sandbox_lab_runs` with `tenant_id`, provider, TTL, and requested lab type.
3. `sandbox-lab-worker` claims the request.
4. The worker deploys temporary lab resources.
5. The worker stores proof in `/api/evidence` compatible tables.
6. The worker creates a sandbox validation job and a cleanup remediation action.
7. For AWS/GCP/Kubernetes labs, the worker queues a real scan job.
8. When the scan finishes, or when TTL expires, the worker destroys the resources and stores cleanup proof.

When the UI sends `demo_scan=true`, the backend forces `scan_after_deploy=true` and `auto_destroy=true`. That keeps the demo button contract simple: create the vulnerable lab, scan it, and delete it automatically.

Supported Labs

Provider	Default state	What it tests	Cost
IaC	Enabled	Public storage, open admin ports, public database, wildcard IAM, CloudFormation exposure, privileged Kubernetes YAML/JSON, wildcard RBAC, exposed services, hardcoded Secret	Zero
AWS	Disabled	Empty S3 bucket with public-access protections disabled, no-MFA IAM user with an inert wildcard policy, unattached security group with public administrative/all-traffic ingress	Near-zero if destroyed quickly
GCP	Disabled	Empty public Cloud Storage bucket and isolated VPC firewall rules exposing administrative ports/all protocols	Near-zero if destroyed quickly
Kubernetes	Disabled	Privileged pod, namespace RoleBinding to `cluster-admin`, permissive/missing NetworkPolicy, NodePort and LoadBalancer services	Uses existing cluster capacity

Required Environment Flags

```
SANDBOX_LABS_ENABLED=true
SANDBOX_LAB_INLINE_WORKER=false
SANDBOX_LAB_BASE_DIR=/app/sandbox-labs
SANDBOX_LAB_WORKER_ID=e2e-sandbox-lab-worker-1
```

```
SANDBOX_LAB_WORKER_POLL_INTERVAL=10

SANDBOX_IAC_DEPLOY=true
SANDBOX_AWS_DEPLOY=false
SANDBOX_AWS_ACCOUNT_IDS=
SANDBOX_GCP_DEPLOY=false
SANDBOX_KUBERNETES_DEPLOY=false
```

Enable real resource creation only in isolated test accounts/projects/clusters:

```
SANDBOX_AWS_DEPLOY=true
SANDBOX_GCP_DEPLOY=true
SANDBOX_KUBERNETES_DEPLOY=true
```

Example API Calls

Zero-cost IaC lab:

```
curl -X POST http://localhost:8000/api/sandbox-labs \
  -H "Content-Type: application/json" \
  -d '{
    "provider": "iac",
    "lab_type": "misconfigured_iac",
    "ttl_minutes": 5,
    "demo_scan": true,
    "scan_after_deploy": true,
    "auto_destroy": true
  }'
```

AWS lab using a credential stored in Manage Credentials:

```
curl -X POST http://localhost:8000/api/sandbox-labs \
  -H "Content-Type: application/json" \
  -d '{
    "provider": "aws",
    "lab_type": "public_s3_and_wildcard_iam",
    "ttl_minutes": 5,
    "credential_id": 12,
    "region": "ap-south-1",
    "demo_scan": true,
    "scan_after_deploy": true,
    "auto_destroy": true
  }'
```

Manual worker trigger:

```
curl -X POST http://localhost:8000/api/sandbox-labs/worker/run-once \
  -H "x-worker-token: $WORKER_TOKEN"
```

Check lab status:

```
curl http://localhost:8000/api/sandbox-labs
```

Destroy a lab manually:

```
curl -X POST http://localhost:8000/api/sandbox-labs/lab-id-here/destroy \
  -H "Content-Type: application/json" \
  -d '{"reason": "demo_finished"}'
```

Credential Requirements

AWS

Use a dedicated sandbox AWS account.

See the AWS demo setup checklist and scoped write-policy template. Set `SANDBOX_AWS_ACCOUNT_IDS` to the approved sandbox account IDs. The worker checks STS identity before provisioning and refuses accounts outside this list and root credentials. Resource IDs are saved incrementally for partial cleanup.

The credential needs permission to:

- call `sts:GetCallerIdentity`
- create/delete the temporary S3 bucket
- update bucket public access block and bucket policy
- create/delete the temporary IAM user and inline policy
- create/delete a temporary VPC and unattached security group
- add public ingress rules to that temporary security group

Keep the permissions scoped to resources prefixed with `cg-` where possible.

GCP

Use a sandbox GCP project. The service account needs permission to:

- create/delete Cloud Storage buckets
- update bucket IAM policy
- create/delete an isolated VPC network
- create/delete temporary firewall rules in that VPC

Kubernetes

Use a sandbox cluster or namespace. The worker container uses the Kubernetes Python client and needs either:

- a stored Kubernetes credential in Manage Credentials, or
- `KUBECONFIG` available in the worker environment

The lab creates a temporary namespace and deletes the namespace during cleanup. The credential also needs permission to create Pods, Services, ServiceAccounts, Roles, RoleBindings, and Secrets inside that namespace. The RoleBinding deliberately grants the lab service account namespace-scoped access through the built-in `cluster-admin` ClusterRole so the scanner can observe the unsafe binding without creating a cluster-wide binding.

Cost Guidance

- IaC lab: no cloud cost.
- AWS/GCP: usually only a few paise/cents for short-lived empty storage resources if cleanup succeeds quickly.
- Kubernetes: no direct cloud API cost, but uses existing cluster CPU/RAM briefly.

Never enable real cloud lab flags in production accounts. Use isolated sandbox accounts/projects and set budget alerts.

Database Tables

- `sandbox_lab_runs`: current state of each lab.
- `sandbox_lab_events`: lifecycle audit trail.
- Existing tables used by the workflow:
  - `scan_jobs`
  - `evidence_artifacts`
  - `security_validation_jobs`
  - `remediation_actions`

Current Limitations

- AWS/GCP/Kubernetes lab creation depends on SDK/CLI availability and the permissions in the selected credential.
- Kubernetes labs require a stored kubeconfig, a `KUBECONFIG` environment variable, or in-cluster configuration in the worker runtime.
- The worker creates intentionally vulnerable resources only inside explicitly enabled sandbox environments.
- The AWS IAM user receives no console password or access key, the security group is not attached to compute, and cloud storage contains no customer data.